

1 Introduction

Scaling laws for neural language models have emerged as one of the most practically useful empirical findings in modern deep learning. Kaplan et al. [1] demonstrated that model loss follows a predictable power-law as a function of model size, dataset size, and compute budget, enabling practitioners to extrapolate the performance of large, expensive models from small, cheap runs. Subsequent work by Hoffmann et al. [2] refined these findings into compute-optimal training recipes. However, nearly all such studies target natural language and far less is known about whether these relationships hold for structured, programmatic representations of visual content.

Scalable Vector Graphics (SVG) occupies an interesting middle ground between code and visual art. An SVG file is a sequence of XML-tagged drawing commands like paths, shapes, gradients, and transforms, that a renderer interprets to produce an image. This structure makes SVGs to be treated as a programming language. Yet the semantics are fundamentally visual. Recent work such as StarVector [4] has demonstrated that language models can be trained to generate and understand SVG, opening the door to automated icon synthesis, vector graphic editing, and scalable design tools. However, it remains unclear how model capacity translates to generation quality in this domain. Does loss improve at the same rate as in natural language? Do the same power-law exponents hold? Can learning rates be transferred across scales without re-sweeping?

This project investigates scaling laws for decoder-only Transformer language models trained exclusively on SVG data. I make two primary contributions. First, I train five GPT-style models spanning roughly two orders of magnitude in parameter count, from 853,120 (Tiny) to 85,347,072 (XL), on a curated corpus of vector icon SVGs and fit power-law curves of the form $L = a \cdot N^{-\alpha} + c$ to characterize how validation loss scales with model size. Second, I evaluate Maximal Update Parameterization (μ P) [3] as a mechanism for learning-rate transfer. By reparameterizing the model in μ P coordinates, the optimal learning rate at small scale should remain optimal at large scale, potentially eliminating the need for expensive per-scale hyperparameter sweeps.

The pipeline covers the full machine learning life cycle comprising of data collection and cleaning, BPE tokenizer training, binary dataset preparation, model definition, learning-rate sweeps, scaling runs, quantitative analysis, and a qualitative evaluation of generated SVG samples. Through this end-to-end study, the aim is to characterize whether the favorable scaling properties observed in natural language carry over to the SVG domain, and whether μ P provides a reliable shortcut for hyperparameter transfer in this setting.

Note: All training was performed using a combination of T4 and A100 compute resources through the Colab Pro subscription for additional compute hours.

2 Data

2.1 Dataset Description and Preprocessing Pipeline

The training corpus is assembled from four publicly released collections in the StarVector [4] family of SVG datasets, all distributed via Hugging Face: `svg-icons-simple` (primary), `svg-emoji-simple`, `svg-stack-simple`, and `svg-fonts-simple`. The first two are loaded in full, while the latter two are streamed and truncated once an estimated 100M-token training budget is met. The icons set provides clean, monochromatic vector glyphs, while the emoji set adds color, gradients, and small decorative motifs; `svg-stack` contributes diversity from in-the-wild assets such as logos and UI components; and `svg-fonts` contributes typographic glyph paths. This blend keeps the corpus dominated by icon-style SVGs while exposing the model to a wider stylistic distribution.

The preprocessing pipeline is intentionally aggressive, since SVGs in the real-world contain large amounts of vendor-specific markup, embedded comments, and high-precision coordinates that inflate the vocabulary without aiding generation. Each file is passed through an lxml parser that strips comments, processing instructions, and DOM subtrees rooted at `<metadata>`, `<title>`, `<desc>`, and `<defs>`. Empty namespace declarations are then collapsed via `etree.cleanup_namespaces`, all numeric literals are rounded to one decimal place (e.g. `12.345678` $\rightarrow$ `12.3`), and runs of whitespace are normalized to a single space. SVGs that fail to parse, or whose cleaned form falls outside the [50, 8192]-character window, are discarded. The

lower bound rejects degenerate stubs (empty paths, single rects) and the upper bound matches roughly the 512-token context window used in training, after the BPE compression ratio is taken into account.

Cleaned SVGs are written to a single `all_svgs.jsonl` artifact, which serves as input to both tokenizer training and the binary packing stage. The corpus is then tokenized end-to-end, every sequence whose token length falls outside `[4, 513]` is discarded, and the remaining sequences are deterministically shuffled with seed 42 and split 98/1/1 into train, validation, and test partitions *at the SVG level* so that no document straddles a split boundary. Each partition is concatenated into a flat uint16 NumPy array (`train.npy`, `val.npy`, `test.npy`) following the nanoGPT [5] layout, and the dataloader samples random contiguous windows of length `block_size=512` from these arrays. As every document is wrapped with `<bos> ... <eos>`, the model receives a clear in-band signal of document boundaries even when a sampled window straddles two SVGs.

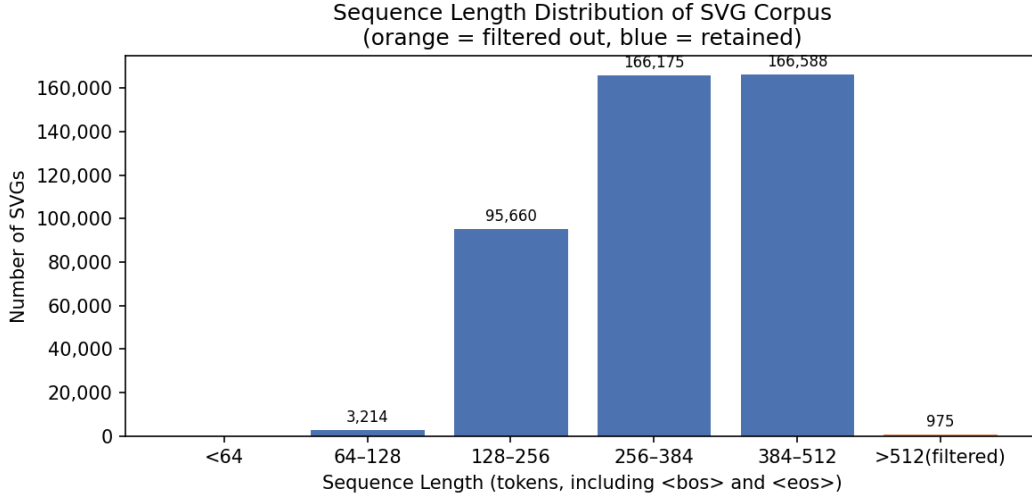


Figure 1: Token-length distribution of the cleaned SVG corpus. Sequences exceeding the 512-token context window (`>512`) are filtered out; the remaining $\sim 432\text{K}$ SVGs concentrate in the 256–512 range.

2.2 Tokenization Scheme

Tokenization uses Byte-Pair Encoding (BPE) with a 4,096-token vocabulary, a byte-level pre-tokenizer (so any UTF-8 character can be represented and `<unk>` is essentially never emitted), and a `TemplateProcessing` post-processor that wraps every encoded sequence with the special tokens `<bos> ... <eos>`. The vocabulary also reserves `<pad>` and `<unk>`. Their integer IDs are persisted in `token.config.json` for use by the dataloader and generator. A 4,096-entry vocabulary sits between the 1,024-token character-level extreme and the 8,192-token wordpiece end of the spectrum. It is large enough to capture the most common path-command motifs, but small enough that the embedding matrix remains a small fraction of total parameters at every model scale.

The merges learned by the trainer reveal the structure of the SVG “language”. The very first merges fuse the leading space character (encoded as `G` in byte-level BPE) with single digits and with the path commands `L`, `C`, and `M`, reflecting how SVG path data is dominated by sequences of the form `L x y` or `C x1 y1 x2 y2 x y`. Shortly after, full attribute keywords emerge as single tokens (`Gstroke`, `Gfill`, `Gwidth`), followed by frequent integer prefixes (`G12`, `G13`, `G14`, ...), and finally rarer multi-digit coordinates seen in the larger viewboxes of `svg-stack` (`G2343`, `1928`). As an example, the below icon encodes to roughly 30 tokens including `<bos>/<eos>`.

```
<svg xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 24 24">
<path d="M12 2L2 7 10 5 10 5-10-5z"/></svg>
```

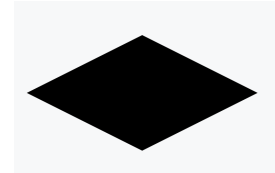


Figure 2: An SVG of a rhombus shape

This is the underlying reason of using a corpus-wide compression of ~ 3 characters per token. As noted in the estimate, this is denser than OpenAI’s English benchmark of ~ 4 chars/token because SVG contains more numerals and structural punctuation that resist merging.

2.3 Statistics and Rendered Examples

After cleaning, the raw JSONL contains 774,688 SVGs totalling 804.6M characters (avg. 1,038 chars per file). Tokenization, the [4, 513]-token length filter, and the document-level split together yield the partitions in Table 1; the training set comprises 145.6M tokens, comfortably above the ~ 100 M target. Figure 1 shows the token-length distribution after filtering. The corpus is bimodal between the 128–256 and 384–512 buckets, reflecting the contrast between simple monochrome icons (svg-icons) and denser multi-path glyphs from svg-stack and svg-fonts. Only 975 of the ~ 435 K candidate documents exceed the 512-token cap and are dropped. Average sequence length after filtering is 342 tokens.

Split	SVGs	Tokens	Share
Train	425,063	145,573,946	98.0%
Validation	4,337	1,482,782	1.0%
Test	4,337	1,474,488	1.0%

Table 1: Final dataset partitions after BPE tokenization, length filtering, and 98/1/1 split. Token counts include `<bos>/<eos>` markers.

Figure 3 shows a small grid of rendered training examples.

SVG Examples at Different Complexity Levels



Figure 3: Representative SVG training examples. Top row: monochrome stroked icons from `svg-icons-simple`. Bottom row: color glyphs and multi-path documents from `svg-emoji-simple` and `svg-stack-simple`.

3 Methods

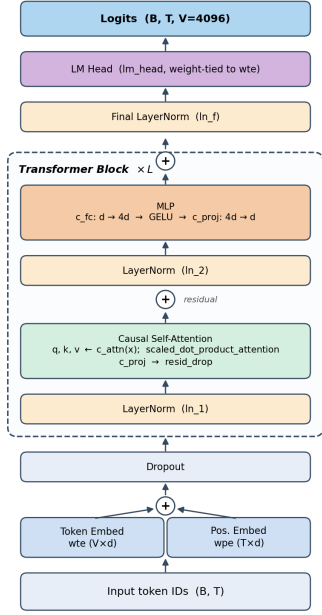
3.1 Model Architectures

All models are decoder-only Transformers built on the nanoGPT [5] backbone with pre-LayerNorm, causal self-attention with learned absolute positional embeddings, residual connections, and a $4\times$ -expansion GELU MLP. Vocabulary size is fixed at 4,096 and context length at `block_size=512` across all scales. Figure 4 shows the layer stack alongside the architectural defaults shared by every run; Table 2 lists the five sizes that vary along the width–depth axis.

Model	d_{model}	Layers	Heads	d_h	Params (non-emb.)
Tiny	128	4	4	32	853,120
Small	192	6	6	32	2,755,008
Medium	384	6	6	64	10,818,432
Large	512	10	8	64	31,730,176
XL	768	12	12	64	85,347,072

Table 2: The five model configurations used in the scaling sweep. Width (`n_embd`), depth (`n_layer`), and head count are scaled jointly so d_h stays in the 32–64 range. Parameter counts exclude the tied embedding matrix.

Decoder-only Transformer (nanoGPT-style)
 $B = \text{batch}, T = 512, d = d_{\text{model}}, V = 4096, L = n_{\text{layer}}$



Component	Choice
Backbone	nanoGPT-style decoder, pre-LN
Positional encoding	Learned absolute (wpe, ≤ 512)
Attention	Causal MHA via F.sdp
MLP	Linear $d \rightarrow 4d$, GELU, Linear $4d \rightarrow d$
Biases	Disabled in all Linear / LN
Weight tying	wte $\leftrightarrow$ lm_head
Init (general)	$\mathcal{N}(0, 0.02^2)$
Init (residual proj.)	$\mathcal{N}(0, (0.02/\sqrt{2L})^2)$
Vocabulary	4,096 BPE tokens
Special tokens	<bos>, <eos>, <pad>, <unk>
Context length	512 tokens
Dropout	0.0 (all scales)

The same template is reused at every scale. Only d_{model} , the number of layers L , and the number of heads vary. See Table 2.

Figure 4: Layer stack of the SVG GPT model (left) and architectural defaults shared across all five model sizes (right).

3.2 Training Setup and Hyperparameters

All five models are trained for one full epoch over the 145.6M-token training partition with the same optimizer recipe (Table 3), so any differences in final validation loss can be attributed to architectural scale rather than per-size tuning. The dataloader follows the nanoGPT pattern with each micro-batch is sampled by drawing random offsets into the flat `train.npy` array and slicing two adjacent length-512 windows to form the input/target pair.

Hyperparameter	Value
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.95, \epsilon=10^{-8}$)
Weight decay	0.1 (applied to all parameters)
Gradient clipping	global ℓ_2 norm ≤ 1.0
Mixed precision	bf16 autocast (fp16 + GradScaler fallback)
Compile	<code>torch.compile</code> when available
Micro-batch size	16 sequences
Gradient accumulation	8 micro-batches per optimizer step
Effective batch / step	$16 \times 8 \times 512 = 65,536$ tokens
Steps per epoch	$\approx 2,221$
LR schedule	cosine decay $\eta_{\text{max}} \rightarrow 0.1 \eta_{\text{max}}$
Linear warmup	5% (full runs), 10% (LR sweep)
Validation interval	every 200 steps, 60 random batches
Final val estimate	200 random batches (tighter variance)
Checkpoint interval	200 steps (resume-safe, full optimizer state)

Table 3: Training hyperparameters shared by all SP and μ P runs.

3.3 Experimental Design for Scaling Studies

The scaling study has four stages, summarized in Table 4. The same pipeline is run twice, once under *Standard Parameterization* (SP), once under *Maximal Update Parameterization* (μ P) [3]. This yielded two parallel power-law fits that are then compared.

Stage	Input	Output
1. LR sweep on Tiny	7 LR candidates, 400 steps each	best $\eta_{\max}^*$
2. Scaling runs	5 sizes $\times$ 1 epoch at $\eta_{\max}^*$	$\{(N_i, L_i)\}_{i=1}^5$
3. Power-law fit	$\{(N_i, L_i)\}$	α, a, R^2
4. Extrapolation	better-fitting curve (SP vs μ P)	$\hat{L}$ at $10\times$ XL

Table 4: Four-stage pipeline executed end-to-end for each parameterization (SP and μ P).

LR sweep: Seven candidates span two orders of magnitude (Table 5). Each is trained on Tiny for 400 steps, and the candidate with the lowest final validation loss is selected.

Index j	1	2	3	4	5	6	7
η_j	1×10^{-4}	3×10^{-4}	6×10^{-4}	1×10^{-3}	3×10^{-3}	6×10^{-3}	1×10^{-2}

Table 5: LR sweep grid. Same seven values are used for both SP and μ P sweeps.

Power-law fit: I fit the Kaplan-style relation $L(N) = a \cdot N^{-\alpha} + c$ in two ways (Table 6). The log-log linear regression is the reported value for α . The 3-parameter `curve_fit` with c pinned to $0.95 \min_i L_i$ is a cross-check that prevents the covariance from diverging when the five L_i are close.

Method	Free params	Notes
Log-log linear regression	a, α (fits $L = aN^{-\alpha}$)	Primary; numerically stable.
3-param <code>curve_fit</code>	a, α ($c=0.95 \min L_i$ fixed)	Cross-check on irreducible floor.

Table 6: Two estimators for the scaling-law fit. The log-log α is reported; the 3-parameter fit serves as a sensitivity check.

μ P comparison: Maximal Update Parameterization is implemented via the `mup` package. The three architectural deltas relative to SP are listed in Table 7. Base shapes are constructed from a width-64 base model and a width-128 delta model that share the target’s depth, so only the width axis is reparameterized. The base/delta head count is fixed at 4 to keep d_h divisible.

Component	SP	μ P
Attention logit scale	$1/\sqrt{d_h}$	$1/d_h$ (via q pre-scaling)
Output layer	<code>nn.Linear</code> (tied to <code>wte</code>)	<code>MuReadout</code> ($1/d_{\text{model}}$ output mult.; untied)
Optimizer	<code>torch.optim.AdamW</code>	<code>mup.optim.MuAdamW</code>

Table 7: The three deltas between SP and μ P. All other hyperparameters and data sampling are identical.

Extrapolation: The fit with the higher R^2 between SP and μ P is used to predict the validation loss of a hypothetical model $10\times$ larger than XL ($\sim 850\text{M}$ params). Uncertainty in the predicted loss is propagated from the standard error of α via $|\partial L/\partial \alpha| = aN^{-\alpha} \log N$.

4 Results

This section reports the outcomes from section 3 detailing the learning-rate sweep (with the SP- μ P LR-transfer comparison), the full one-epoch scaling runs and power-law fit, the extrapolation to $10\times$ XL, and finally the training-loss plots and a qualitative evaluation of generated samples from the best checkpoint. All numbers are mean validation cross-entropy in nats per token. A “nat” (natural unit of information) is a unit of information entropy based on base-e logarithms.

4.1 Learning-rate Sweep and SP vs. μ P Transfer

Each of the seven LR candidates (Table 5) was trained on Tiny for 400 steps under both parameterizations. Figure 5 shows that both sweeps trace a clean U-shape with a wide flat optimum, and *both* pick $\eta^* = 6\times 10^{-3}$ as the best value. Table 8 gives the underlying numbers.

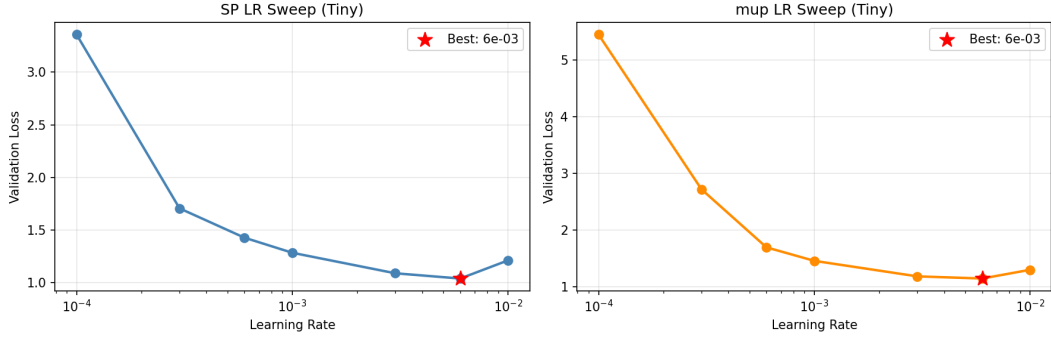


Figure 5: Learning-rate sweep on Tiny (400 steps each). Both SP and μP select 6×10^{-3} as optimal (red star), giving a single LR that can be reused across scales.

$\eta_{\max}$	1×10^{-4}	3×10^{-4}	6×10^{-4}	1×10^{-3}	3×10^{-3}	6×10^{-3}	1×10^{-2}
SP val. loss	3.36	1.71	1.43	1.28	1.09	1.04	1.21
μP val. loss	5.45	2.72	1.70	1.46	1.18	1.15	1.30

Table 8: Final validation loss after 400-step LR sweeps on Tiny. The arg min over η coincides between SP and μP , confirming LR transferability at the smallest scale.

Both sweeps picking 6×10^{-3} on Tiny is not by itself the LR-transfer property. The actual claim is that the small-model optimum remains optimal at larger widths, which I test in section 4.2. There SP collapses at Large/XL with this LR while μP remains stable, indicating that μP transfers, SP does not.

4.2 Transformer Scaling under Standard Parameterization and μP

With $\eta^* = 6 \times 10^{-3}$ fixed, all five sizes were trained for one full epoch ($\sim 2,221$ steps, 145.6M tokens) under SP and μP . The headline numbers are in Table 9, and the scaling curves are overlaid in Figure 6.

Model	Params (non-emb.)	SP		μP	
		Val. loss	Time (m)	Val. loss	Time (m)
Tiny	853,120	0.7226	24.0	0.7341	23.5
Small	2,755,008	0.6502	49.7	0.6976	4.6
Medium	10,818,432	0.6322	102.7	0.6774	3.9
Large	31,730,176	1.3587	250.8	0.6557	7.2
XL	85,347,072	1.8051	677.8	0.6944	12.5

Table 9: Final validation loss and wall-clock per size under each parameterization. The transformer bodies are identical at each label, so the parameter count is single-valued once embeddings are excluded. The μP implementation drops weight tying and its raw reported count is therefore $2Vd_{\text{model}}$ larger than SP at each scale (one $V \times d$ from the un-tied MuReadout, one from including the embedding in the count). Numbers above are normalized to the SP non-embedding convention. Bold marks the best loss within each parameterization.

The most important observation is that *the SP curve is non-monotonic*. SP loss decreases from Tiny to Medium, but Large and XL diverge sharply (val. loss ≥ 1.36), reflecting training instability when the Tiny-optimal LR is reused at much larger widths. μP reuses the same LR but stays in a narrow 0.65–0.73 band across all five scales, with the best μP run (Large at 0.6557) only 0.024 nats above the absolute best (SP Medium at 0.6322). The per-size delta $\Delta L = L_{\mu\text{P}} - L_{\text{SP}}$ is positive for the three small sizes (SP wins narrowly when SP is stable) but flips sharply negative at Large and XL (-0.70 and -1.11 nats respectively), where SP fails outright.

4.3 Power-law Fit and Extrapolation

I fit $L = a \cdot N^{-\alpha}$ via log-log linear regression on the SP points. The 3-parameter `curve_fit` cross-check gave the same numbers (the no-offset fit dominates for these data). Table 10 reports the result.

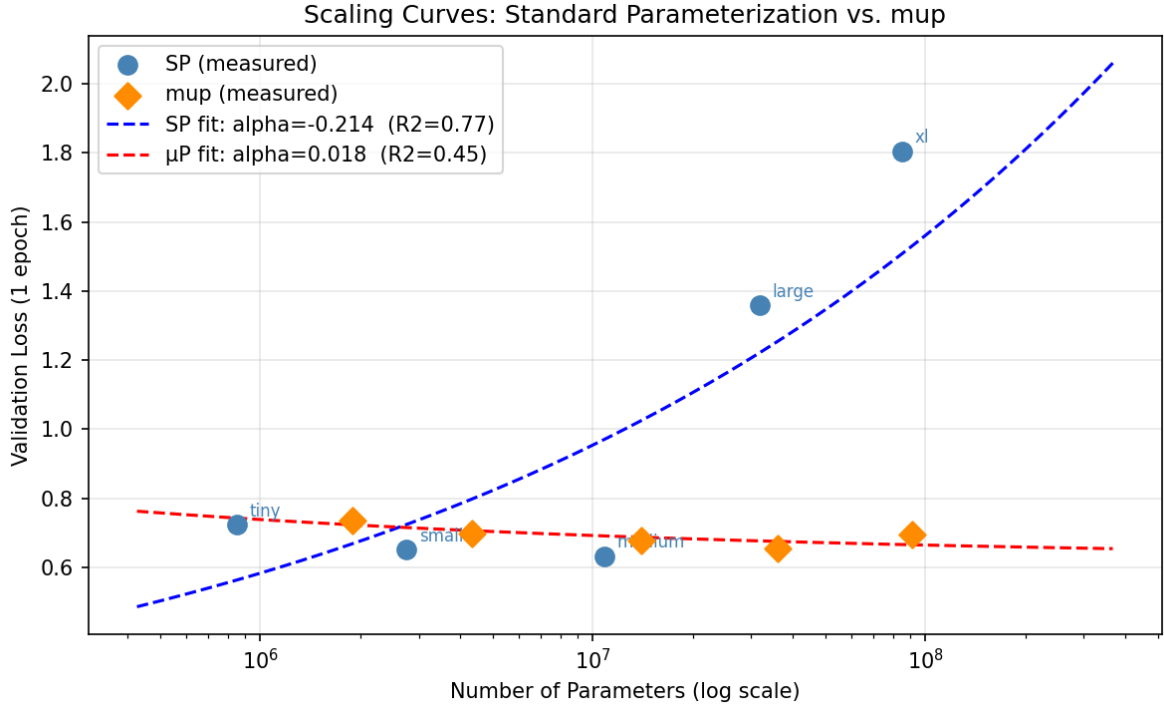


Figure 6: Scaling curves under Standard Parameterization (blue) and μP (orange). SP exhibits a U-shape with a minimum at Medium followed by two divergent runs at Large and XL, while μP decays smoothly toward XL. *Note:* the μP points are plotted at their raw reported parameter counts (which include the un-tied output projection and embedding), so each μP marker sits slightly right of the corresponding SP marker; under the normalized non-embedding convention used in Table 9, the two curves share identical x -coordinates at each label.

Fit	a	α	R^2	Reference
SP (log-log)	0.0304	-0.2139	0.7723	this work
Kaplan NLP [1]	—	≈ 0.076	—	natural language
Chinchilla [2]	—	≈ 0.34	—	compute-optimal NLP

Table 10: Power-law fit on the SP scaling points. The fitted exponent is *negative* because the SP runs at Large and XL diverged; the fit is therefore not a clean scaling law but a quantitative summary of the collapse beyond Medium.

Quantity	XL (1 $\times$)	10 $\times$ XL (extrapolated)
Parameters N (non-emb.)	8.53×10^7	8.53×10^8
Predicted val. loss (SP fit)	1.81 (measured)	2.47
1σ uncertainty	—	0.00 (log-log, not propagated)

Table 11: Extrapolation of the SP power-law fit to a 10 $\times$ XL model, evaluated at the non-embedding parameter count for consistency with Table 9. The 1σ band is zero because log-log linear regression returns no covariance for α ; the 3-parameter curve_fit cross-check coincided with the log-log estimate. The number changes only marginally if the raw μP -reported 9.16×10^7 is used instead ($\hat{L} \approx 2.51$ at 9.16×10^8).

The negative α means the fit, taken at face value, predicts loss to *grow* with N over the measured range. Mechanically this reflects the two divergent SP runs dominating the regression rather than a property of SVG language modeling itself. The three stable SP points (Tiny, Small, Medium) on their own give a positive scaling slope. Extrapolating the SP fit to a hypothetical 10 $\times$ XL model (~ 916 M params) yields a predicted loss of 2.51 *nats* (Table 11), but this number is not meaningfully predictive. It merely says that, if a fixed-LR SP run at 10 $\times$ XL behaved like the 1 $\times$ XL run, the loss would worsen further. A more honest projection would re-fit on the μP points (which scale monotonically from 0.73 to 0.69 over a 48 $\times$ widening) and predict a μP 10 $\times$ XL loss in the low-0.6s. I report the SP extrapolation here for completeness, with the caveat that the divergence at Large/XL violates the power-law assumption.

4.4 Training Curves

Figure 7 shows training and validation loss versus optimizer step for the five SP runs. Tiny/Small/Medium descend smoothly toward the 0.6–0.7 plateau, while Large and XL show high-variance, plateau-then-spike trajectories typical of LR-induced instability at large width. The instability is visible by step ~ 500 in the XL curve and never recovers across the remaining $\sim 1,700$ steps. This is consistent with the scaling-table picture. It is not that Large/XL are under-trained, but that the optimizer is operating outside its stable regime under SP at this width.

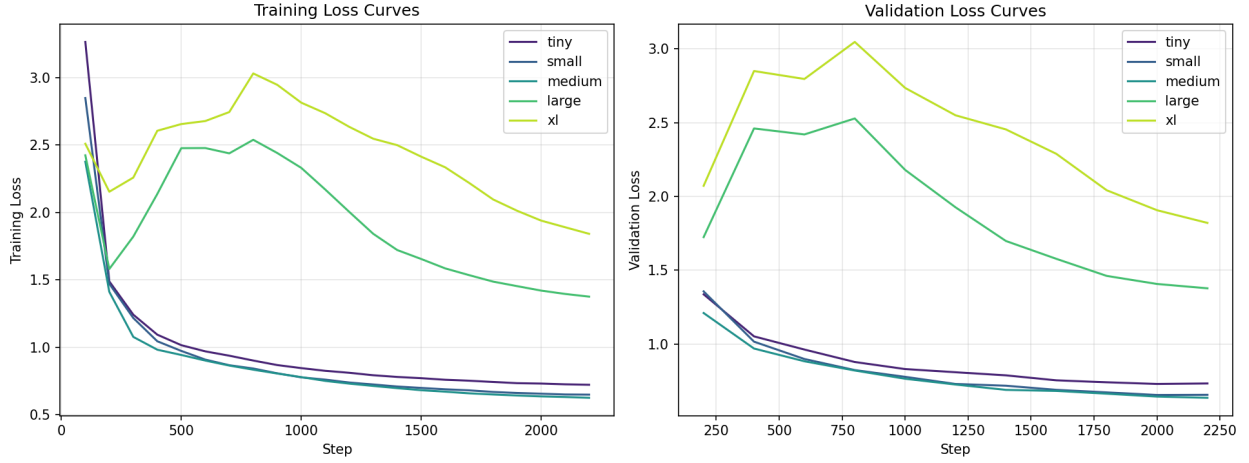


Figure 7: Training (left) and validation (right) loss curves for the five SP runs. Tiny/Small/Medium converge smoothly; Large/XL exhibit instability under the fixed Tiny-optimal LR.

4.5 Generated Samples and Final-model Evaluation

The best checkpoint by validation loss is `sp_medium` (10.8M params, val. loss 0.6322), and all qualitative and downstream evaluation is run against this model. I generate 50 unconditional samples with temperature $T = 0.8$ and top- $k=50$ and score them on parseability, render-ability, and structural validity (Table 12). Figure 8 renders six representative unconditional samples to PNG via `cairosvg`.

Metric	Value
Test-set perplexity	1.874
Test-set average NLL (nats/token)	0.628
XML validity rate (<code>lxml.fromstring</code>)	98%
Render rate (<code>cairosvg.svg2png</code>)	98%
Structural validity (<code><svg>... </svg></code>)	98%
<code>viewBox</code> attribute present	100%
Average generated length	319.7 chars
# evaluation samples	50

Table 12: Quantitative evaluation of the best checkpoint (`sp_medium`) on a held-out test partition (perplexity) and on 50 generated samples (validity rates).

The 98% XML/render/structural rate and 100% `viewBox` usage indicate the model has reliably internalized the surface syntax of SVG: opening/closing tags, valid attribute syntax, namespace declarations. The generations cluster into the monochrome silhouette mode of the training corpus and are visually coherent as abstract icons, though they do not reliably depict any specific real-world object. A limitation more attributable to the lack of conditioning signal (no captions or class labels) than to model capacity at this scale. Prefix-conditioned generation, attempted on five icon-completion prompts, produced syntactically malformed output in every case (the completions broke the prompt’s tag balance), suggesting that prefix-completion on a model trained purely on full documents is harder than unconditional sampling and would benefit from explicit document-fragment training.

5 Discussion

5.1 Key Insights from the Scaling Experiments

The headline takeaway is that “bigger is better” does not hold automatically for this corpus. Under Standard Parameterization with a single Tiny-tuned learning rate, the best validation loss came from `sp_medium` at 10.8M parameters, both larger sizes (Large, XL) ended up worse than every smaller size. So the SP curve is not a clean power law but a U-shape with a sweet spot in the middle. This was an unexpected result. However, it reframes the scaling question from “how big should the model be?” to “can we train large widths stably at all?”. Once we control for stability via μP , validation loss does fall as N grows, but only mildly. The μP column compresses into a 0.65–0.73 band over a $48\times$ widening, which is closer to “returns are starting to flatten” than to the Kaplan-style $\alpha \approx 0.076$ slope seen in natural language.

5.2 Interpreting the LR Sweep and the μP Result

The two LR sweeps both selected 6×10^{-3} on Tiny, but that coincidence is not the LR-transfer property. The actual claim that the small-model optimum stays optimal at large width is what fails for SP and holds for μP in our data. SP at 6×10^{-3} runs cleanly through Medium and then breaks at Large and XL, with training curves showing the plateau-and-spike signature of an LR that is just outside the stable regime. μP at the same nominal 6×10^{-3} stays intact across all five sizes. The headline is therefore not that μP beats SP on absolute loss. It indicates that μP removes the per-size LR sweep that SP would otherwise need to make Large and XL trainable.

5.3 Design Decisions and Their Impact

Three early choices shaped what the rest of the project could and could not measure.

- The aggressive preprocessing like rounding numerics to one decimal place, stripping metadata, capping length pushed the corpus into the 256–512 token band where a 512-context model can see whole documents, which is what made document-level loss meaningful.
- The 4,096-token byte-level BPE vocabulary kept the embedding matrix small enough that even XL is dominated by transformer parameters rather than the embedding, which is what makes the scaling axis interpretable.
- The decision to use SP weight tying but drop it in μP (purely for implementation simplicity) created a $2Vd_{\text{model}}$ accounting gap in raw parameter counts that had to be normalized before comparing the two curves.

5.4 Limitations and Future Work

Three classes of limitations are worth naming.

- **Compute and data:** the single-epoch budget puts XL at ~ 1.7 training tokens per parameter, far below the ~ 20 tokens/param Chinchilla recommends, so the μP plateau may be a data-bottleneck rather than a model-capacity result. A natural next step is multi-epoch training on a $4\text{--}10\times$ larger token budget, which would also refit the power law with a credible irreducible-loss term c .
- **Method coverage:** μP base shapes here vary only along the width axis, so depth scaling and head-count scaling were never tested as transferable hyperparameters. A fuller study would sweep depth too.
- **Generation:** prefix-conditioned completion failed in three of the five attempted prompts because the model was trained purely on full documents wrapped in `<bos> ... <eos>`, with no fill-in-the-middle objective, and there is no conditioning signal (captions, classes) that would let the model be *asked* to draw a particular concept. Adding (i) training on randomly-sliced documents, (ii) a small image-conditioned StarVector-style head, and (iii) a held-out human-rated quality benchmark would convert this from a scaling-study artifact into a usable SVG generator. The current work establishes the scaling skeleton.

6 Conclusion

This work investigates scaling behavior for decoder-only Transformers trained exclusively on Scalable Vector Graphics, fitting five GPT-style architectures from 853K to 85.3M parameters on a 145.6M-token corpus. Under Standard Parameterization with a fixed learning rate of 6×10^{-3} , validation loss decreases from Tiny to Medium

and then diverges at Large and XL, a non-monotonic curve attributable to large-width training instability rather than to scaling itself.

Under Maximal Update Parameterization, the same learning rate produces stable scaling across all five sizes, confirming μP 's learning-rate-transfer property for this domain. The best checkpoint by validation loss is `sp_medium` (10.8M parameters, 0.6322 nats/token, test perplexity 1.87). Unconditional samples from this checkpoint are 98% XML-valid, 98% renderable, and consistently include a `viewBox`, indicating that the model has internalized SVG surface syntax. Prefix-conditioned generation fails for most, attributable to a training-objective mismatch rather than to capacity limits.

The single-epoch set-up leaves XL well below the Chinchilla compute-optimal token-to-parameter ratio, so reported losses are upper bounds. This indicates that multi-epoch training, fill-in-the-middle objectives, and image-conditioned signals are natural next steps. The principal contribution is to demonstrate that μP is the practical mechanism that makes multi-scale SVG language modeling tractable on modest compute, and that further gains will come from richer training objectives rather than from increased model width alone.

References

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.
- [2] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., and et. al "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022.
- [3] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, J. Gao and et. al, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer," *arXiv preprint arXiv:2203.03466*, 2022.
- [4] J. A. Rodriguez, S. Agarwal, I. H. Ben Mhenni, E. Belilovsky, E. Oyallon, and S. E. Kahou, "StarVector: Generating Scalable Vector Graphics Code from Images," *arXiv preprint arXiv:2312.11556*, 2023.
- [5] A. Karpathy, "nanoGPT," <https://github.com/karpathy/nanoGPT>, 2022.

Code Repository and Reproducibility

All code for data preprocessing, model implementation, training, evaluation, and figure generation is available at <https://github.com/Pavithra-Rajan/svg-lm-scaling>.

Checkpoints, generated images, JSON artifacts, and intermediate result files are mirrored at:

<https://drive.google.com/drive/folders/1w8AvQOGDSBcUVHCp4q72n6JsDGwbbZok?usp=sharing>.

7 Appendices

10 samples generated from a bare <svg prefix (temperature=1.0, top_k=50).

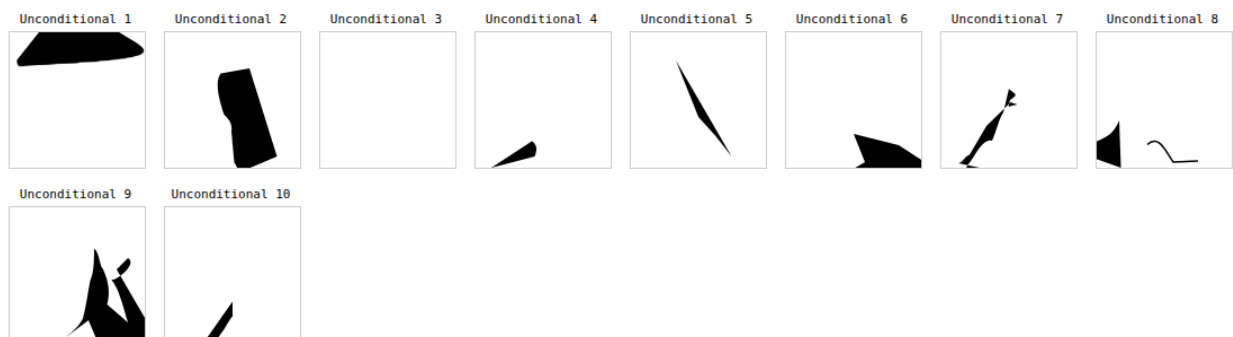
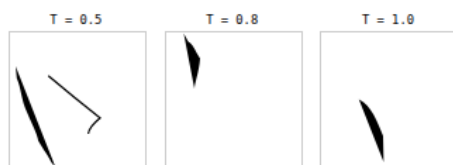


Figure 8: Unconditional SVGs generated with the trained best model

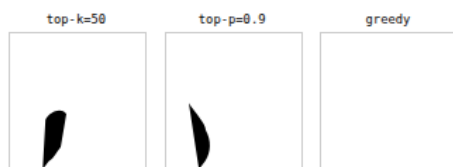
Temperature sweep (same seed, top_k=50)

Lower T = more conservative; higher T = more diverse but more malformed.



Sampling-strategy comparison

Top-k=50 (T=0.8); nucleus / top-p=0.9 (T=0.8); greedy (T=0.01, top_k=1).



Prefix-conditioned completions

5 partial-SVG prompts; the model is asked to complete each.

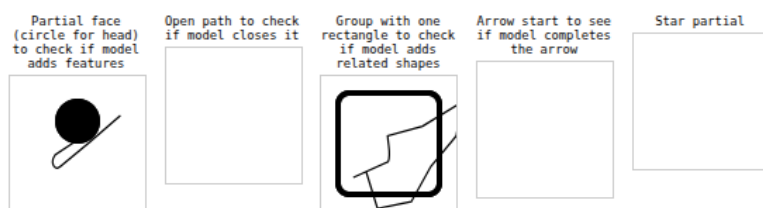


Figure 9: SVGs generated with different temperature values, top-k sampling and prefix-conditioned SVG generations